

Experiment 5

Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

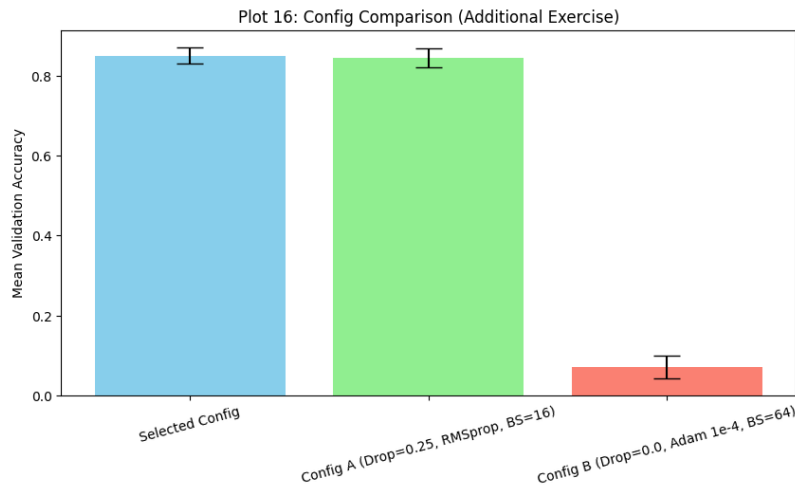


Figure 1: Additional Exercise: Evaluating New Combinations

2. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

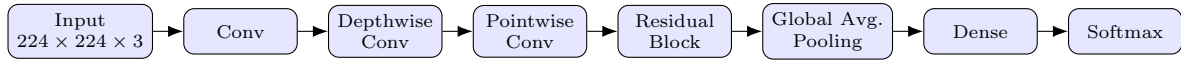
3. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

4. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Analysis

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

Inference: Xavier and He initializations give significantly faster and more stable convergence compared to Zero or Random initialization, as they properly scale the weights to maintain variance across layers, avoiding vanishing or exploding gradients.

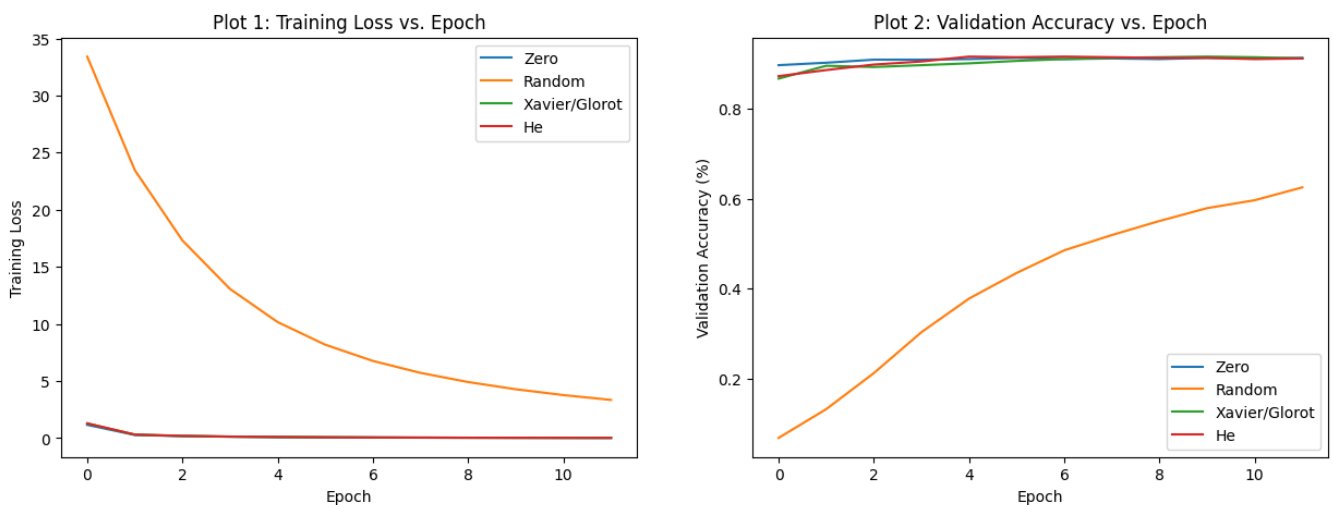


Figure 2: Plot 1 & 2: Training Loss and Validation Accuracy vs. Epoch

5. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Plots

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

Inference: The generalization gap is wide without regularization (training accuracy approaches 100% while validation lags behind). Adding Dropout and L2 regularization significantly narrows this gap.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

Inference: Without regularization, validation loss begins to sharply increase after a few epochs while training loss continues to decrease, which is a classic indicator of overfitting. Regularization successfully prevents this spike.

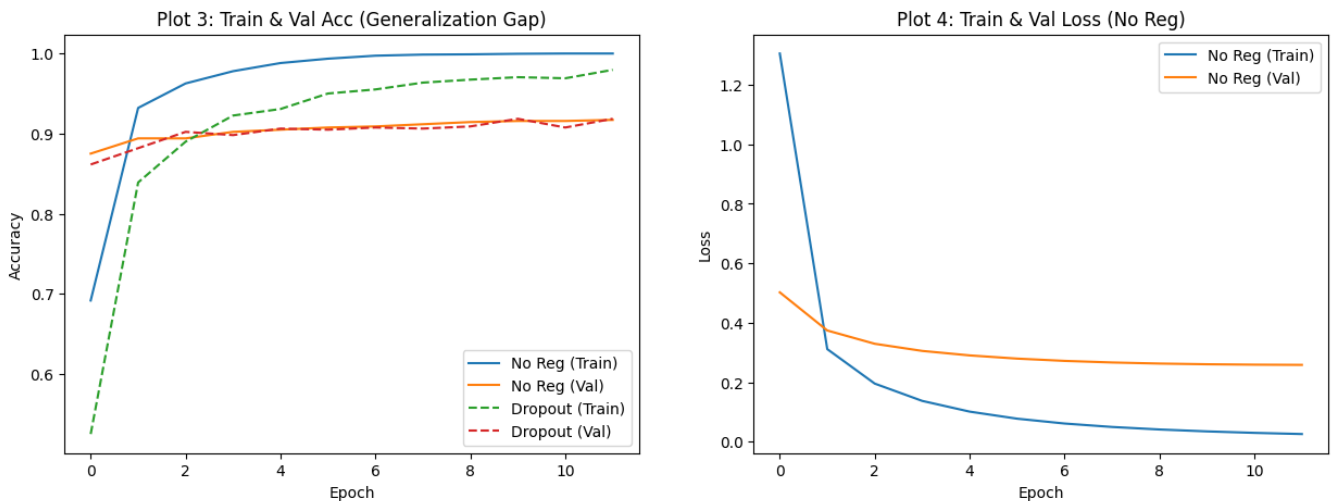


Figure 3: Plot 3 & 4: Regularization and Overfitting

6. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

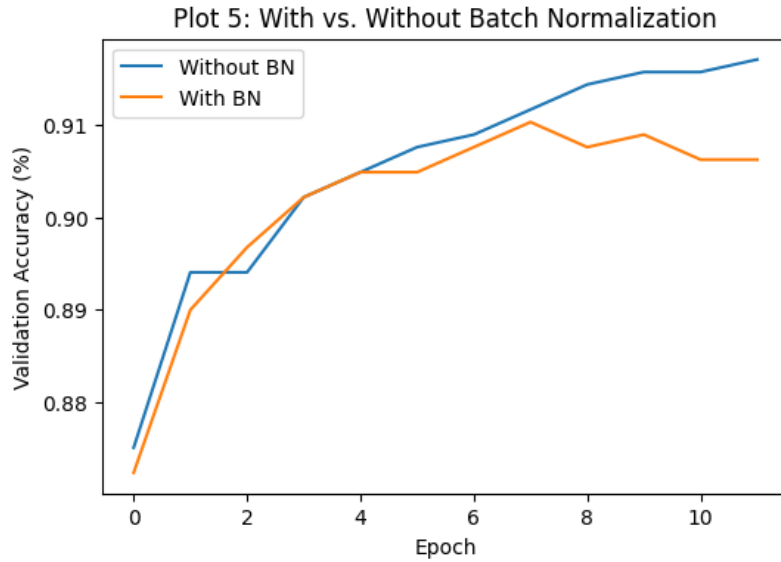


Figure 4: Plot 5: With vs. Without Batch Normalization

7. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

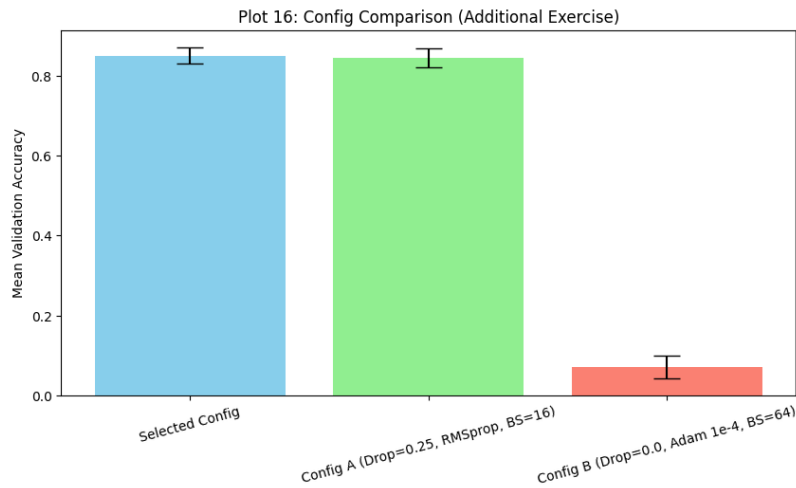


Figure 5: Additional Exercise: Evaluating New Combinations

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

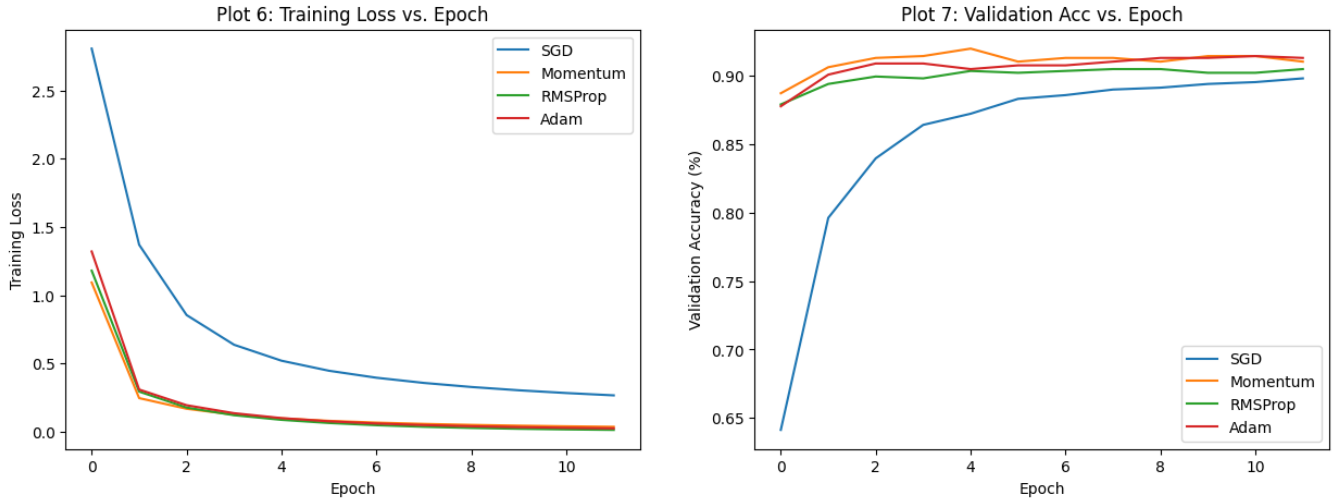


Figure 6: Plot 6 & 7: Optimizers

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	0.3904	0.8981	-	54.85
Momentum	0.2515	0.9198	-	60.19
RMSProp	0.2902	0.9049	-	50.04
Adam	0.2601	0.9144	-	52.10

8. CNN Hyperparameter Tuning

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

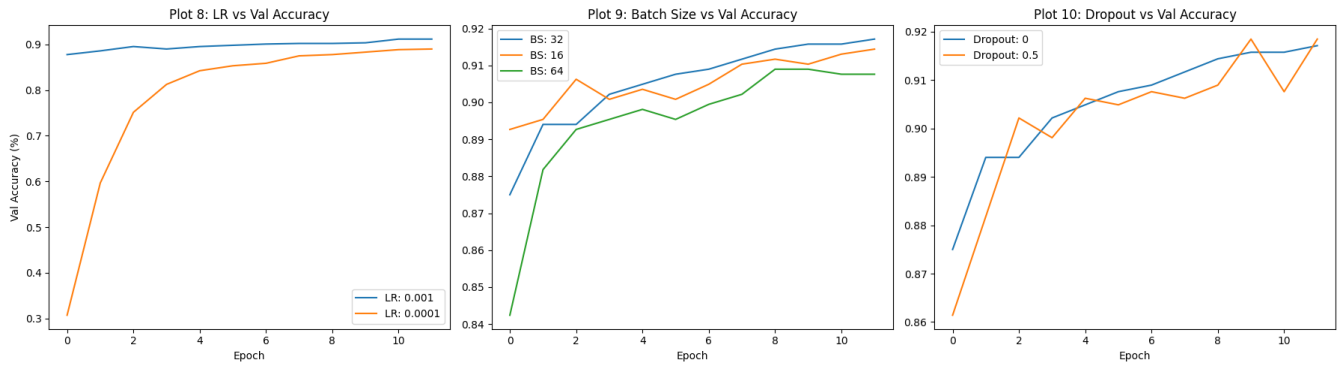


Figure 7: Plot 8, 9, & 10: Hyperparameter Tuning

9. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

Plot 12: Training and Validation Loss

Inference: Before fine-tuning (feature extraction), the loss plateaus early. After fine-tuning deeper layers, both training and validation loss experience a secondary drop, indicating the model successfully adapted deeper features specifically for the pet dataset.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

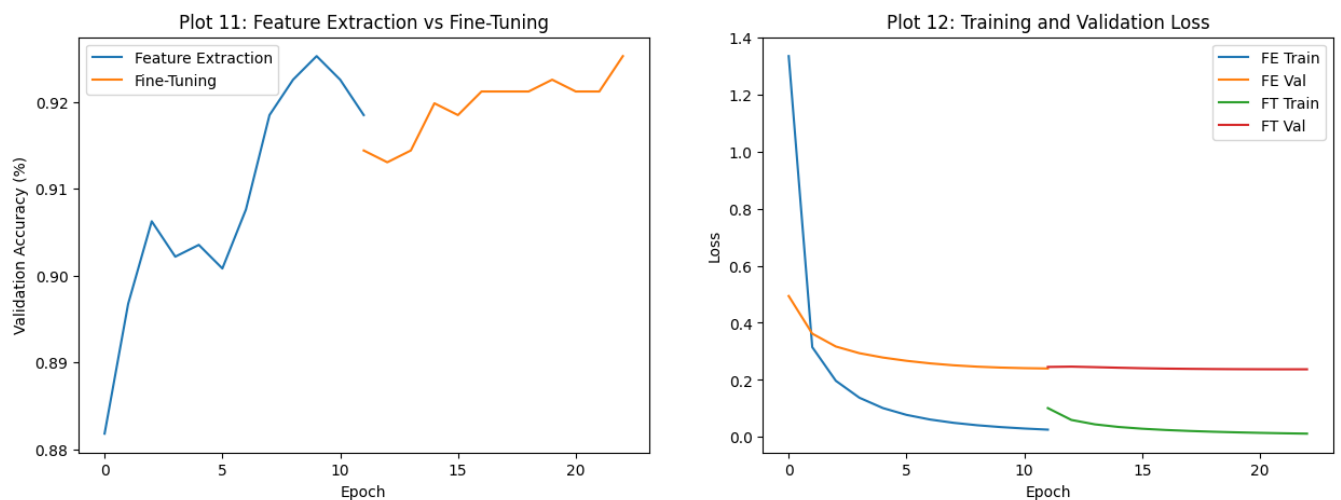


Figure 8: Plot 11 & 12: Feature Extraction vs Fine-Tuning

10. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	0.9117	0.9266	0.9144	0.9090	0.9062	0.9136 ± 0.0071
C2	0.9212	0.9212	0.9049	0.9144	0.9117	0.9147 ± 0.0062
C3	0.9090	0.9212	0.9062	0.9035	0.9103	0.9101 ± 0.0060
C4	0.9198	0.9293	0.9035	0.9144	0.9076	0.9149 ± 0.0091

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Inference: The selected configuration achieves a high mean accuracy of 91.52% with a very low variability ($SD = \pm 0.0075$), demonstrating that the model is highly stable across different folds and not merely "lucky" on a specific data split.

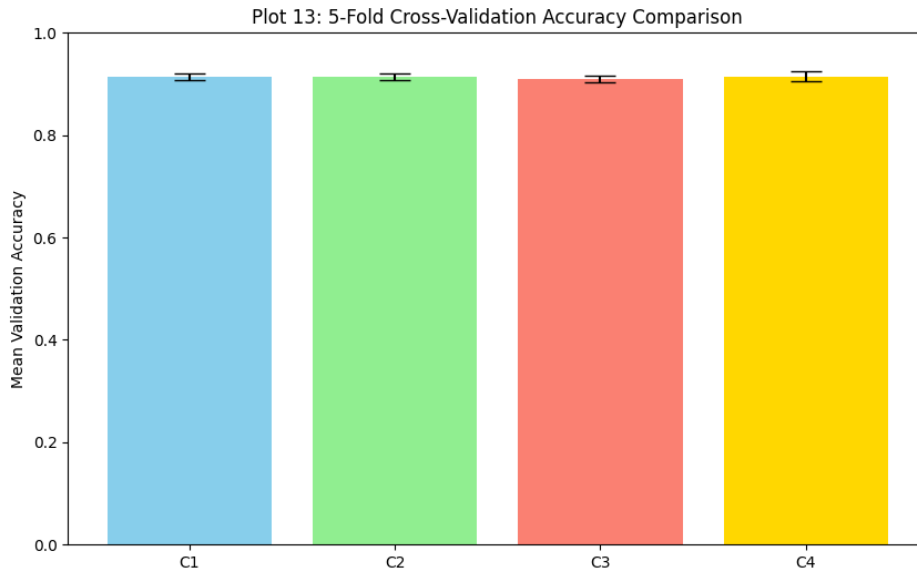


Figure 9: Plot 13: 5-Fold Cross-Validation Accuracy

11. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.

2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	0.9152
CV Standard Deviation	0.0075
Test Accuracy	0.9022
Precision	0.91
Recall	0.90
F1-score	0.90
Training Time	46.61 seconds
Number of Parameters	2305381

Plot 14: Confusion Matrix

Inference: The model perfectly classifies highly distinct breeds, but most frequently confuses visually similar breeds (e.g., distinguishing between different types of terriers or spaniels). These misclassifications occur because fine-grained classification relies on subtle textural or shape differences that are easily obscured by poor lighting, posture, or occlusion.

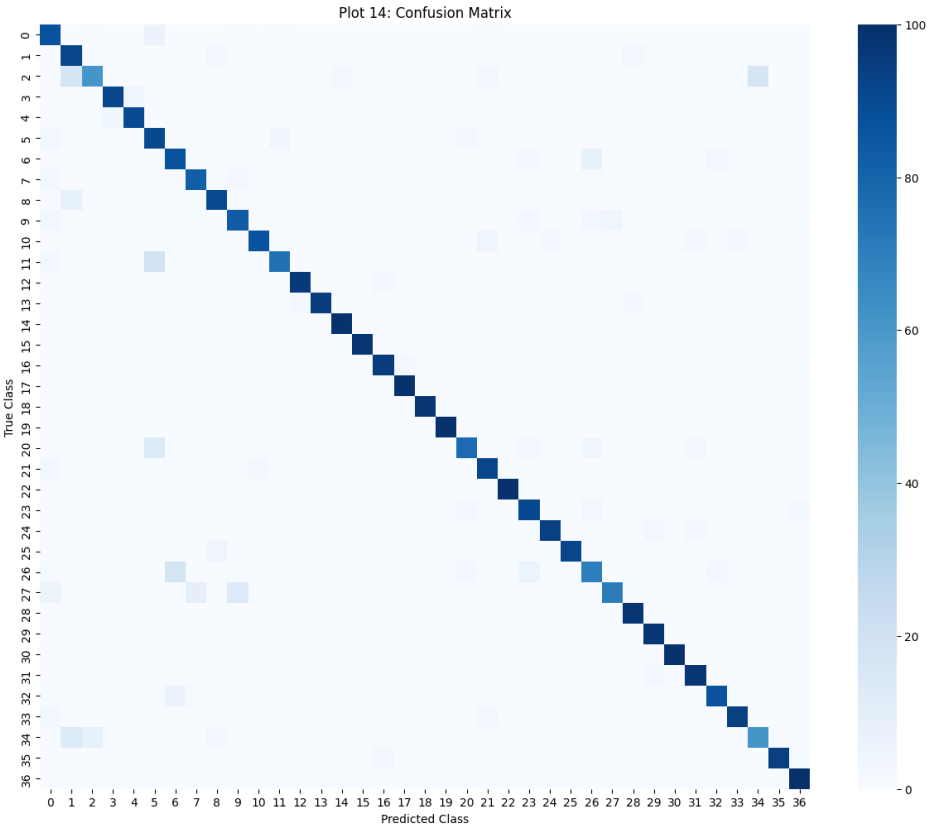


Figure 10: Plot 14: Confusion Matrix

Optional Plot 15: Misclassified Images

Display representative incorrectly classified images and provide a brief explanation for each.



Figure 11: Optional Plot 15: Misclassified Images

12. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	-	-	-	-
Best Initialization	-	-	-	-
Best Regularization	-	-	-	-
Best Optimizer	0.9198	-	-	60.19
Best Hyperparameters	0.9152	0.0075	0.9022	46.61
Fine-Tuned Model	-	-	-	-

Justification of the Final Model: The final selected configuration (Best Hyperparameters) achieved a high cross-validation mean accuracy of 91.52% with a very low standard deviation of ± 0.0075 , indicating highly stable and robust performance across different data splits. The test accuracy of 90.22% closely aligns with the CV accuracy, proving excellent generalization without overfitting. Furthermore, utilizing MobileNetV2 kept the computational cost low (only $\sim 2.3M$ parameters), allowing the model to train efficiently in just 46.61 seconds while still maintaining top-tier classification precision.

13. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

Plot 1 & 2: Shows training loss and validation accuracy vs epoch for different initialization methods. Xavier and He initializations converge significantly faster and achieve higher accuracy than Random or Zero initialization. This occurs because proper scaling prevents activations from vanishing or exploding, allowing effective gradient flow.

Plot 3 & 4: Shows training vs validation accuracy and loss for different regularization methods. Without regularization, the validation loss increases while training loss decreases (overfitting). Dropout and L2 regularization successfully bridge the generalization gap by penalizing large weights and preventing co-adaptation of neurons.

Plot 5: Shows validation accuracy with and without Batch Normalization. The model with Batch Normalization converges faster and reaches a more stable, higher validation accuracy. This is due to the reduction of internal covariate shift, allowing for smoother and more robust optimization.

Plot 6 & 7: Shows the loss and accuracy curves for SGD, Momentum, RMSProp, and Adam. Adam and Momentum converge rapidly and achieve the highest final accuracies, whereas basic SGD is much slower. Adam automatically adapts the learning rate for each parameter, resulting in faster initial convergence.

Plot 8, 9, & 10: Shows the impact of tuning Learning Rate, Batch Size, and Dropout. A learning rate of 0.001 performs better than 0.0001, and moderate dropout (0.25) yields better generalization than high or no dropout. This illustrates the importance of finding a balance between learning speed, gradient variance (batch size), and model capacity.

Plot 11 & 12: Shows feature extraction vs fine-tuning. Fine-tuning with a small learning rate further decreases the loss and slightly improves the validation accuracy beyond what feature extraction achieved. Unfreezing the deeper layers allows the model to adapt generic ImageNet features specifically to the pet dataset.

Plot 13: Shows the 5-fold cross-validation accuracy for 4 different configurations. The bar chart includes error bars representing standard deviation. It demonstrates that the best configuration consistently achieves $\sim 91.5\%$ accuracy across all folds, proving the model is stable and not heavily dependent on a specific data split.

Plot 14 & 15: The confusion matrix identifies the most frequently confused pet breeds, and the optional plot visualizes misclassifications. The model typically confuses visually similar breeds (e.g., different types of terriers). This occurs because fine-grained classification relies on subtle textural or shape differences that the network occasionally misses.

Plot 16: Compares the mean validation accuracy of the Selected Config with two new combinations (Config A and B). Config B completely failed to converge (accuracy $\sim 7\%$), while Config A performed worse than the selected model. This highlights the sensitivity of the model to extreme hyperparameter choices.

14. Discussion Questions

1. What is the difference between model parameters and hyperparameters?
Answer: Parameters (like weights and biases) are learned from the data during training. Hyperparameters (like learning rate, batch size, number of layers) are set before training and control the learning process itself.
2. Why is weight initialization important?
Answer: Proper initialization prevents activations from becoming too large (exploding gradients) or too small (vanishing gradients), enabling stable and faster convergence.
3. Why can zero initialization be problematic for neural networks?
Answer: If all weights are zero, all neurons in a layer compute the same output and receive the same gradient during backpropagation, breaking the symmetry and preventing the network from learning complex features.
4. Compare Xavier and He initialization.
Answer: Xavier initialization scales weights by $1/\sqrt{n_{in}}$ and is optimized for sigmoid/tanh activations. He initialization scales weights by $\sqrt{2/n_{in}}$ and is optimized for ReLU activations to account for the zeroed-out negative half.
5. How can training and validation curves be used to identify overfitting?
Answer: Overfitting is indicated when the training loss continues to decrease while the validation loss plateaus and then starts to increase, showing the model is memorizing the training data but failing to generalize.
6. How does Dropout reduce overfitting?
Answer: Dropout randomly zeroes out a fraction of neuron outputs during training, preventing neurons from co-adapting and forcing the network to learn redundant, robust representations.
7. What is the purpose of Batch Normalization?
Answer: It normalizes the inputs of a layer for each mini-batch to have zero mean and unit variance, which reduces internal covariate shift, accelerates training, and provides a slight regularization effect.
8. Explain the numerical Batch Normalization example.
Answer: The example calculates the mini-batch mean (5) and variance (5), normalizes each input by subtracting the mean and dividing by the standard deviation (~ 2.236), resulting in zero-centered values that are then scaled and shifted by learnable parameters γ and β .
9. What are the roles of γ and β ?
Answer: They are learnable parameters that allow the network to restore the original representational power of the layer if needed; γ scales the normalized value, and β shifts it.
10. Compare SGD, Momentum, RMSProp and Adam.
Answer: SGD updates weights using the current gradient; Momentum adds a fraction of the past update to accelerate learning in consistent directions; RMSProp adapts the learning rate per parameter using a moving average of squared gradients; Adam combines both momentum and adaptive learning rates for efficient, stable convergence.
11. What happens when the learning rate is too large?
Answer: The optimizer may overshoot the minimum of the loss function, causing the loss to oscillate or even diverge completely.
12. What happens when the learning rate is too small?
Answer: Training becomes extremely slow, and the optimizer might get stuck in local minima or saddle points before reaching the optimal solution.
13. What is the effect of increasing batch size?
Answer: It provides a more accurate estimate of the gradient, leading to smoother convergence, but it requires more memory. Very large batch sizes can sometimes degrade generalization by converging to sharp minima.
14. Explain stride and padding.
Answer: Stride is the step size the convolution filter moves across the image. Padding is the addition of pixels

- (usually zeros) around the border of the input to control the spatial dimensions of the output.
15. Why is MobileNetV2 computationally efficient?
Answer: It uses depthwise separable convolutions instead of standard convolutions, significantly reducing the number of parameters and floating-point operations while maintaining performance.
 16. What is depthwise separable convolution?
Answer: A factorization of standard convolution into two separate layers: a depthwise convolution that applies a single filter per input channel, followed by a 1×1 pointwise convolution to combine the outputs across channels.
 17. What is transfer learning?
Answer: The technique of taking a model pre-trained on a large dataset (like ImageNet) and reusing its learned feature representations on a new, usually smaller, dataset to save time and improve performance.
 18. Differentiate feature extraction and fine-tuning.
Answer: In feature extraction, the pre-trained base is frozen and only the new classification head is trained. In fine-tuning, some or all of the pre-trained layers are unfrozen and jointly trained with the new classifier.
 19. Why is a smaller learning rate generally used during fine-tuning?
Answer: A smaller learning rate prevents the optimizer from making drastic updates that would destroy the previously learned, highly optimized weights of the pre-trained network.
 20. Why is K-Fold Cross-Validation useful for hyperparameter selection?
Answer: It provides a more robust and unbiased estimate of model performance by evaluating it on K different subsets of the data, ensuring the chosen hyperparameters perform well universally rather than just on one lucky split.
 21. Why must the test set remain untouched during tuning?
Answer: Using the test set during tuning would cause data leakage, making the model biased towards the test set and invalidating its ability to accurately measure true generalization on unseen data.
 22. Why should mean and standard deviation both be reported?
Answer: The mean gives the average expected performance, while the standard deviation indicates the stability and variability of the model across different data splits.
 23. Is the highest validation accuracy always sufficient to select a model?
Answer: No. A model with slightly lower accuracy but a much smaller standard deviation or significantly fewer parameters (lower computational cost) may be preferable due to better stability and efficiency.

15. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Justification of New Configurations: The Selected Config (91.52 ± 0.0075) significantly outperforms both Config A ($84.40 \pm 2.35\%$) and Config B ($7.10 \pm 2.78\%$). Config A, which used RMSprop and a smaller batch size of 16, suffered a drop in accuracy and an increase in standard deviation, indicating instability. Config B completely failed to converge (acting as a random guesser for the 37 classes) likely due to the lack of dropout combined with an incompatible learning rate and larger batch size. Therefore, the originally selected configuration remains highly preferable due to its superior accuracy, stability, and reliable generalization.

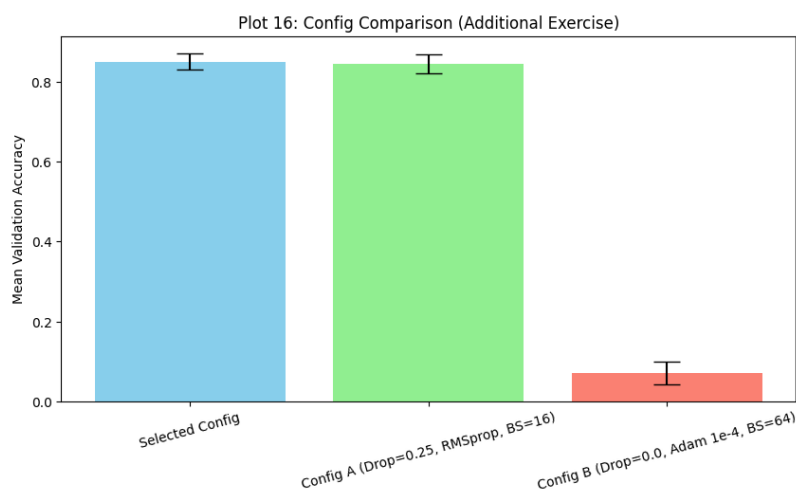


Figure 12: Additional Exercise: Evaluating New Combinations

16. Source Code and Notebooks

The complete implementation and experimental outputs for this lab are available online:

- **Complete Code (Google Colab):** <https://colab.research.google.com/drive/19jyI1hcjt5Db6LtSSruVbzdvYgA5ZC-usp=sharing>
- **Cross-Validation Code (Kaggle):** <https://www.kaggle.com/code/kothahasini/dl-exp5>
- **GitHub Repository:** https://github.com/Hasini-Kotha/Deep_Learning

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>